

**Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»
Факультет Программной Инженерии и Компьютерной Техники**

Блок задач с Timus
Алгоритмы и Структуры Данных

Выполнил:
Ларионов Владислав Васильевич
Группа Р3209

Проверил:
Косяков Михаил Сергеевич

Санкт-Петербург, 2026

Содержание

1444. Накормить слепотама	3
Условие:	3
Решение:	4
О решении:	6
1726. Кто ходит в гости.....	7
Условие:	7
Решение:	8
О решении:	9

1444. Накормить элѣфпотама

Условие:

Гарри Поттер сдаёт экзамен по предмету «Уход за магическими существами». Его задание — накормить карликового элѣфпотама. Гарри помнит, что элѣфпотамы отличаются прямолинейностью и невозмутимостью. Они настолько прямолинейны, что ходят строго по прямой, и настолько невозмутимы, что заставить их идти можно, только если привлечь его внимание к чему-нибудь действительно вкусному. И главное, наткнувшись на цепочку своих собственных следов, элѣфпотам впадает в ступор и отказывается идти куда-либо. По словам Хагрида, элѣфпотамы обычно возвращаются домой, идя в обратную сторону по своим собственным следам. Поэтому они никогда не пересекают их, иначе могут заблудиться. Увидев свои следы, элѣфпотам детально вспоминает все свои перемещения от выхода из дома (поэтому-то они и ходят только по прямой и лишний раз не меняют направление — так легче запоминать). По этой информации элѣфпотам вычисляет, в какой стороне расположена его нора, после чего поворачивается и идет прямо к ней. Эти вычисления занимают у элѣфпотама некоторое (довольно большое) время. А то, что некоторые невежды принимают за ступор, на самом деле есть проявление выдающихся вычислительных способностей этого чудесного, хотя и медленно соображающего животного!

Любимое лакомство элѣфпотамов — слоновьи тыквы, именно они и растут на лужайке, где Гарри должен сдавать экзамен. Перед началом испытания Хагрид притащит животное к одной из тыкв. Скормив элѣфпотаму очередную тыкву, Гарри может направить его в сторону любой оставшейся тыквы. Чтобы сдать экзамен, надо провести элѣфпотама по лужайке так, чтобы тот съел как можно больше тыкв до того, как наткнется на свои следы.

Исходные данные

В первой строке входа находится число N ($3 \leq N \leq 30000$) — количество тыкв на лужайке. Тыквы пронумерованы от 1 до N , причем номер один присвоен той тыкве, у которой будет стоять элѣфпотам в начале экзамена. В следующих N строках даны координаты всех тыкв по порядку. Все координаты — целые числа от -1000 до 1000 . Известно, что положения всех тыкв различны, и не существует прямой, проходящей сразу через все тыквы.

Результат

В первой строке выхода вы должны вывести K — максимальное количество тыкв, которое может съесть элѣфпотам. Далее по одному числу в строке выведите K чисел — номера тыкв в порядке их обхода. Первым в этой последовательности всегда должно быть число 1.

Пример

исходные данные	результат
4 0 0 10 10 0 10 10 0	4 1 3 2 4

Решение:

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

struct Point {
    int x, y;
};

vector<Point> points;
double cx, cy;

int get_quadrant(double x, double y) {
    if (y >= 0 && x > 0) return 0;
    if (y > 0 && x <= 0) return 1;
    if (y <= 0 && x < 0) return 2;
    return 3;
}

bool compare(int a, int b) {
    double ax = points[a].x - cx;
    double ay = points[a].y - cy;
    double bx = points[b].x - cx;
    double by = points[b].y - cy;

    int qa = get_quadrant(ax, ay);
    int qb = get_quadrant(bx, by);
    if (qa != qb) {
        return qa < qb;
    }

    double mul = ax * by - ay * bx;
    if (mul != 0) {
        return mul > 0;
    }

    double dist_a = ax * ax + ay * ay;
    double dist_b = bx * bx + by * by;

    return dist_a < dist_b;
}

int main() {
    int N;
    cin >> N;

    points.resize(N + 1);

    long long sum_x = 0, sum_y = 0;
    for (int i = 1; i <= N; i++) {
```

```

        cin >> points[i].x >> points[i].y;
        sum_x += points[i].x;
        sum_y += points[i].y;
    }

    cx = (double)(sum_x) / N;
    cy = (double)(sum_y) / N;

    vector<int> order(N);
    for (int i = 0; i < N; i++) {
        order[i] = i + 1;
    }

    sort(order.begin(), order.end(), compare);

    int start = 0;
    for (int i = 0; i < N; i++) {
        if (order[i] == 1) {
            start = i;
            break;
        }
    }

    cout << N << "\n";
    for (int i = 0; i < N; i++) {
        int idx = (start + i) % N;
        cout << order[idx] << "\n";
    }
}

```

О решении:

Идея:

Благодаря условию, что все точки не могут быть расположены на одной прямой мы можем сделать вывод, что есть хотя бы одна точка вне этой прямой. Тогда точки образуют собой многоугольник. Расположим в центре масс всех точек новую точку – центральную. Тогда из этой точки до каждой можно провести вектор. Тогда эти вектора не будут пересекаться. Таким образом можно отсортировать эти точки по углу относительно центра масс, находящегося внутри всех точек, а потом эти точки соединить, образовав структуры звезды. Потом подгоняем массив так, чтобы первой шла точка с индексом 1, и задача решена.

Сложность по времени:

$O(n \log n)$ – так как сортируем массив точек (метод `sort` реализует `introsort`)

Сложность по памяти:

$O(n)$ – так как храним массив точек.

1726. Кто ходит в гости...

Условие:

Программный комитет школьных соревнований по программированию, проходящих в УрГУ — многочисленная, весёлая и дружная команда. Дружная настолько, что общения в университете им явно не хватает, поэтому они часто ходят друг к другу в гости. Все ребята в программном комитете очень спортивные и ходят только пешком.

Однажды хранитель традиций олимпиадного движения УрГУ подумал, что на пешие прогулки от дома к дому члены программного комитета тратят слишком много времени, которое могли бы вместо этого потратить на придумывание и подготовку задач. Чтобы доказать это, он решил посчитать, какое расстояние в среднем преодолевают члены комитета, когда ходят друг к другу в гости. Хранитель традиций достал карту Екатеринбурга, нашёл на ней дома всех членов программного комитета и выписал их координаты. Но координат оказалось так много, что хранитель не смог справиться с этой задачей самостоятельно и попросил вас помочь ему.

Город Екатеринбург представляет собой прямоугольник со сторонами, ориентированными по сторонам света. Все улицы города идут строго с запада на восток или с севера на юг, проходя через весь город от края до края. Дома всех членов программного комитета расположены строго на пересечении каких-то двух перпендикулярных улиц. Известно, что все члены комитета ходят только по улицам, поскольку идти по тротуару гораздо приятнее, чем по дворовым тропинкам. И, конечно, при переходе от дома к дому они всегда выбирают кратчайший путь. Программный комитет очень дружный, и все его члены ходят в гости ко всем одинаково часто.

Исходные данные

Первая строка содержит целое число n — количество членов программного комитета ($2 \leq n \leq 10^5$). В i -й из следующих n строк через пробел записаны целые числа x_i, y_i — координаты дома i -го члена программного комитета ($1 \leq x_i, y_i \leq 10^6$).

Результат

Выведите среднее расстояние, которое проходит член программного комитета от своего дома до дома своего товарища, округлённое вниз до целых.

Пример

исходные данные	результат
3 10 10 20 20 10 20	13

Решение:

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

long long sum_by_dir(vector<long long> vector_dir) {
    sort(vector_dir.begin(), vector_dir.end());
    long long impact = 0, prefix_sum = 0;

    for (int i = 0; i < (int)vector_dir.size(); i++) {
        impact += vector_dir[i] * i - prefix_sum;
        prefix_sum += vector_dir[i];
    }

    return 2LL * impact;
}

int main() {
    int houses_count;
    cin >> houses_count;

    vector<long long> dist_x(houses_count), dist_y(houses_count);
    for (int i = 0; i < houses_count; i++) {
        cin >> dist_x[i] >> dist_y[i];
    }

    long long sum_x = sum_by_dir(dist_x);
    long long sum_y = sum_by_dir(dist_y);
    long long total_sum = sum_x + sum_y;

    cout << total_sum / (1LL * houses_count * (houses_count - 1));
}
```


О решении:

Идея:

Если можно перемещаться только по улицам, а они перпендикулярны, то расстояние лесенкой от эквивалентно сумме сторон прямоугольника, где в противоположных вершинах лежат дома. Чтобы найти суммарное расстояние, пройденное между всеми парами домов, найдем суммарное расстояние по x , затем по y , сложим и поделим на количество не уникальных пар. Чтобы не считать расстояние между каждой парой домов за квадрат, воспользуемся префиксной суммой. Итеративно проходя по массиву координат, для каждого i -го элемента массива координат в суммарное расстояние, пройденное всеми по x будем прибавлять сумму расстояний, пройденных от i -го дома до всех прошлых домов.

Сложность по времени:

$O(n \log n)$ – так как используем сортировку.

Сложность по памяти:

$O(n)$ – так как храним в памяти два массив координат.